

Speech Emotion Recognition

61.501 Production Ready ML

Sagar Pratap Singh | 1010736 Seow Chun Yong | 1000108 Yang Qingwei | 1010737

Motivation & AI for Social Good

Why Speech Emotion Recognition Matters

Emotions are central to human communication, but machines mostly “hear words”, not feelings



Support tools for
counselling / Helplines



Workplace
Environment



Elderly Support



Teacher / Student
Learning Environment

Our Goal: Use only classical ML models from class to build a practical **speech emotion recognizer**

What The Project Is About

Recognising six emotions from short speech recordings

Objective



Classify 6 emotions from short audio clips **angry, happy, sad, fear, disgust, neutral**

Key Challenges



- Acoustic overlap between emotions:
 - **Angry & happy** → both loud, fast (high arousal)
 - **Fear & sad** → both quiet, hesitant (low arousal)
- Emotions are subjective
- Limited quality dataset

Our Task

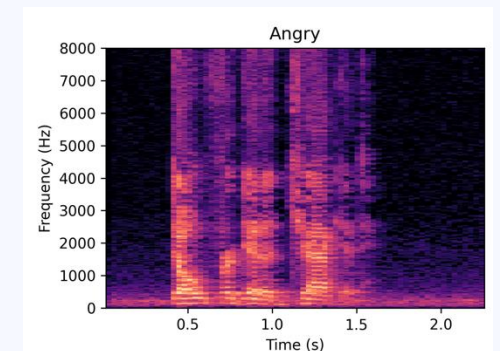
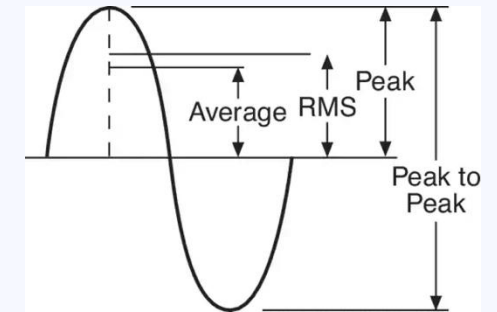
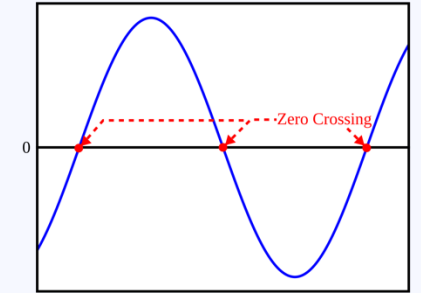


Find acoustic features and classifiers that can separate these overlapping categories

Standard Audio Features [1/2]

Understanding Acoustic Descriptors

1. **Zero Crossing Rate (ZCR):** The rate at which the audio signal changes from positive to negative (or vice versa).
2. **Root Mean Square Energy (RMS):** A measure of the signal's loudness/energy, computed as the square root of the mean of squared amplitude values.
3. **Mel Spectrogram:** A representation of how energy is distributed across different frequencies over time, scaled to match human hearing perception.



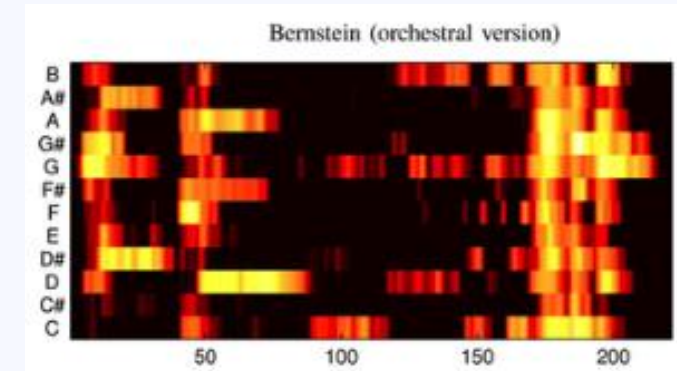
Standard Audio Features [2/2]

Understanding Acoustic Descriptors

4. Chroma Features:

Energy distribution across the 12 pitch classes of Western music regardless of octave.

(C, C#, D, D#, E, F, F#, G, G#, A, A#, B)



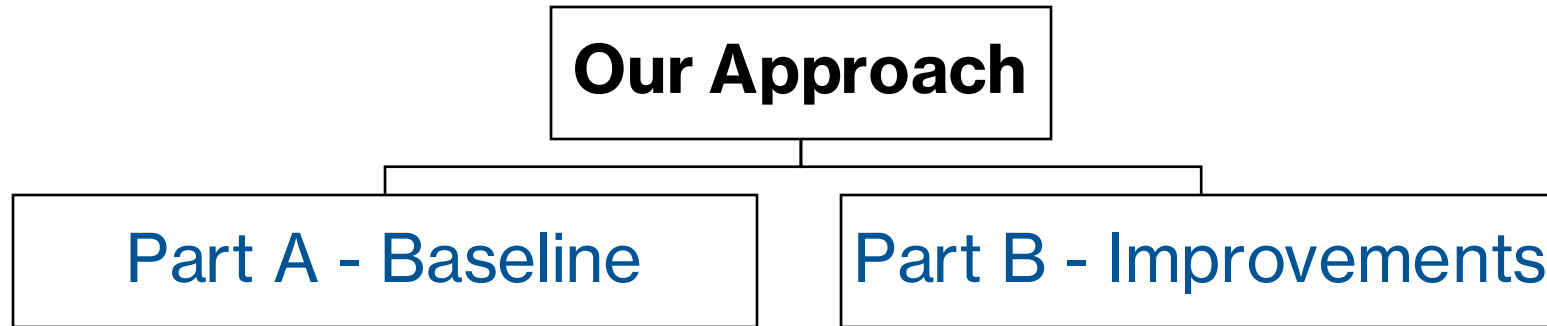
5. Mel-Frequency Cepstral Coefficients (MFCCs):

Capture the shape of the vocal tract (how the mouth, tongue and throat are positioned), which determines the "color" or timbre of the voice.



Our Approach

Crowd-sourced Emotional Multimodal Actors Dataset



Establish a baseline using
standard features
(162 features)

Systematically improved upon baseline
through **enhanced feature engineering**
(210 features), **hyperparameter tuning**
and **ensemble methods**

Exploring the Dataset: CREMA-D

Crowd-sourced Emotional Multimodal Actors Dataset

Original Dataset

- **7,442 clips** recorded by **91 actors** (48 male, 43 female).
- Age range **20-74**, multiple ethnic backgrounds.
- Contains controlled emotional expressions in different intensities.

Data Augmentation

- We **tripled** the dataset (to 22,326) to **improve generalisation** and **robustness**
- We added
 - **Random Noise:** simulates microphone and environmental noise.
 - **Time Stretch + Pitch Shift:** simulates different speakers and emotional intensities.

Part A – Baseline Models

Nine classical models evaluated using 162 features

Purpose: Establish a clear baseline for further improvements

Logistic
Regression

Decision Tree

Random Forest

SVM
(Linear)

SVM
(Polynomial)

SVM
(RBF)

Naïve Bayes

K-Nearest
Neighbours

Bagging

Part A – Baseline Performance & Insights [1/4]

Nine classical models evaluated using 162 features – What Worked Well

Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

← Best Model

← Worst Model



Training Process

- 1. Data Splitting**
75% training, 25% testing
- 2. Feature Scaling**
Standard Scaler
- 3. Default Settings**
Sklearn Model Default

Part A – Baseline Performance & Insights [2/4]

Nine classical models evaluated using 162 features – What Worked Well

Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

★ What Worked Well

✓ **Tree-based bagging** ensembles are the best performing (**58.7% / 56.5%**)

1. Nested Conditions: Emotions have complex combination of multiple features.

Decision tree can capture this well:

"if RMS > 0.5 AND mel_band_80 > 0.3 AND pitch_range > 50 Hz, then predict angry."

Part A – Baseline Performance & Insights [3/4]

Nine classical models evaluated using 162 features – What Worked Well

Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

★ What Worked Well

✓ **Tree-based bagging** ensembles are the best performing (**58.7% / 56.5%**)

2. Noisy Data: Ensemble averaging **guards against noise**. A single tree might overfit, but when 100 trees vote, the noise gets averaged out.

3. Insensitive to feature scaling: Our features span very different numeric ranges

Part A – Baseline Performance & Insights [4/4]

Nine classical models evaluated using 162 features – What Worked Well

Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

★ What Worked Well

- ✓ Bagging Classifier performed slightly better (58.7%) than Random Forest (56.5%)
 - Bagging uses **all features** at each split
 - RF considers random subset of features

Part A – Baseline Performance & Insights [1/5]

Nine classical models evaluated using 162 features – What Remains Challenging

What Remains Challenging

! Some model families are fundamentally mismatched to the task

➤ Naïve Bayes achieved (23.1%)

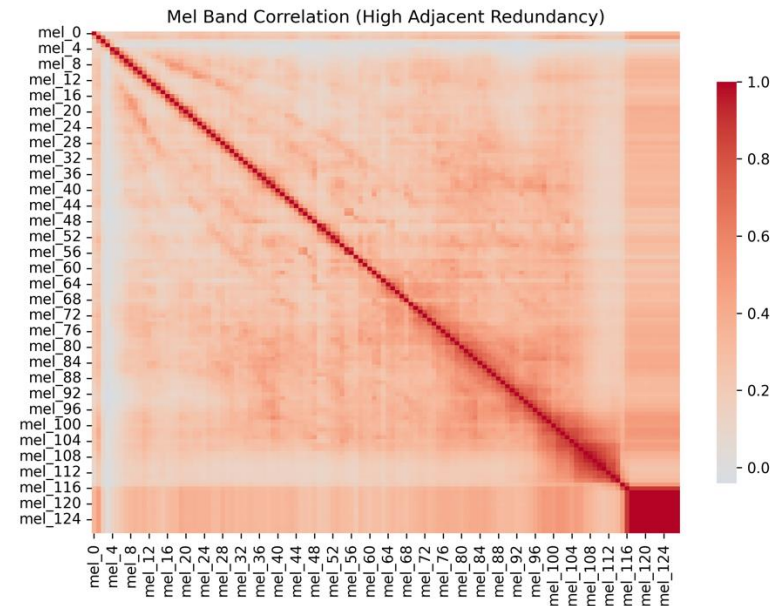
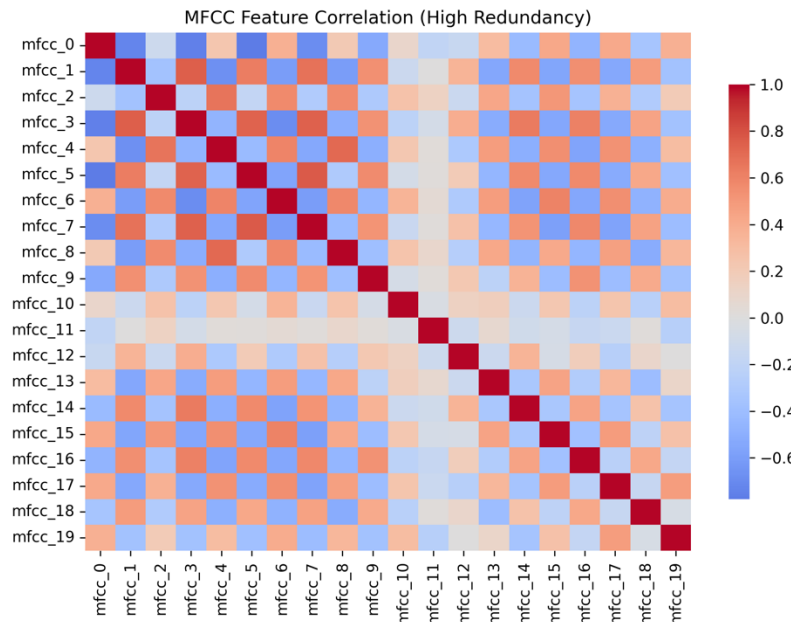
Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

Part A – Baseline Performance & Insights [2/5]

Some model families are fundamentally mismatched to the task

How real-world audio patterns conflict with Naïve Bayes' core assumptions.

- Many features were strongly correlated (MFCCs: 0.6–0.8, Mel bands: >0.9), **violating Naïve Bayes' independence assumption**
- Models like **tree ensembles**, **SVMs** and **logistic regression** handled these correlations well, leading to much higher accuracy.



Part A – Baseline Performance & Insights [3/5]

Nine classical models evaluated using 162 features – What Remains Challenging

What Remains Challenging

- ! **Some model families are fundamentally mismatched to the task**
 - Naïve Bayes achieved (23.1%)
- ! **Error patterns reveal deeper challenges**
 - Bagging Classifier (58.7%) correctly classified **six out of ten** test samples on average, but still confused **four out of ten**.
 - Different classifier made **different-kinds of mistakes**.

Model	Original Test Accuracy	Training Time
Bagging Classifier	58.7%	379.52s
Random Forest (Bagging)	56.5%	57.54s
SVM (RBF)	47.4%	54.84s
SVM (Polynomial)	36.5%	12.50s
SVM (Linear)	45.4%	90.15s
Logistic Regression	43.8%	0.63s
Decision Tree	43.6%	0.47s
K-Nearest Neighbours	38.1%	54.41s
Naive Bayes (Gaussian)	23.1%	0.07s

Part A – Baseline Performance & Insights [4/5]

Error patterns reveal deeper challenges

Observation 1: Model confuses between these emotion pairs: Fear-Sad, Angry-Happy, Disgust-Sad

- ! **Errors were not random**, they clustered around **specific emotion pairs**
- ! Certain emotions are **acoustically similar**:
 - **fear and sad** both involve **quiet, hesitant speech**
 - **angry and happy** both involve **loud, energetic speech**

We need to **design features** or classifiers **specifically target** these confused pairs.

Bagging Classifier

angry	727	49	27	118	24	8
disgust	83	472	26	110	140	
fear	118	76	386	108	179	
happy	153		54	528	84	41
neutral	11	101	35	82	473	113
sad	5	72	63	14	109	691
	angry	disgust	fear	happy	neutral	sad

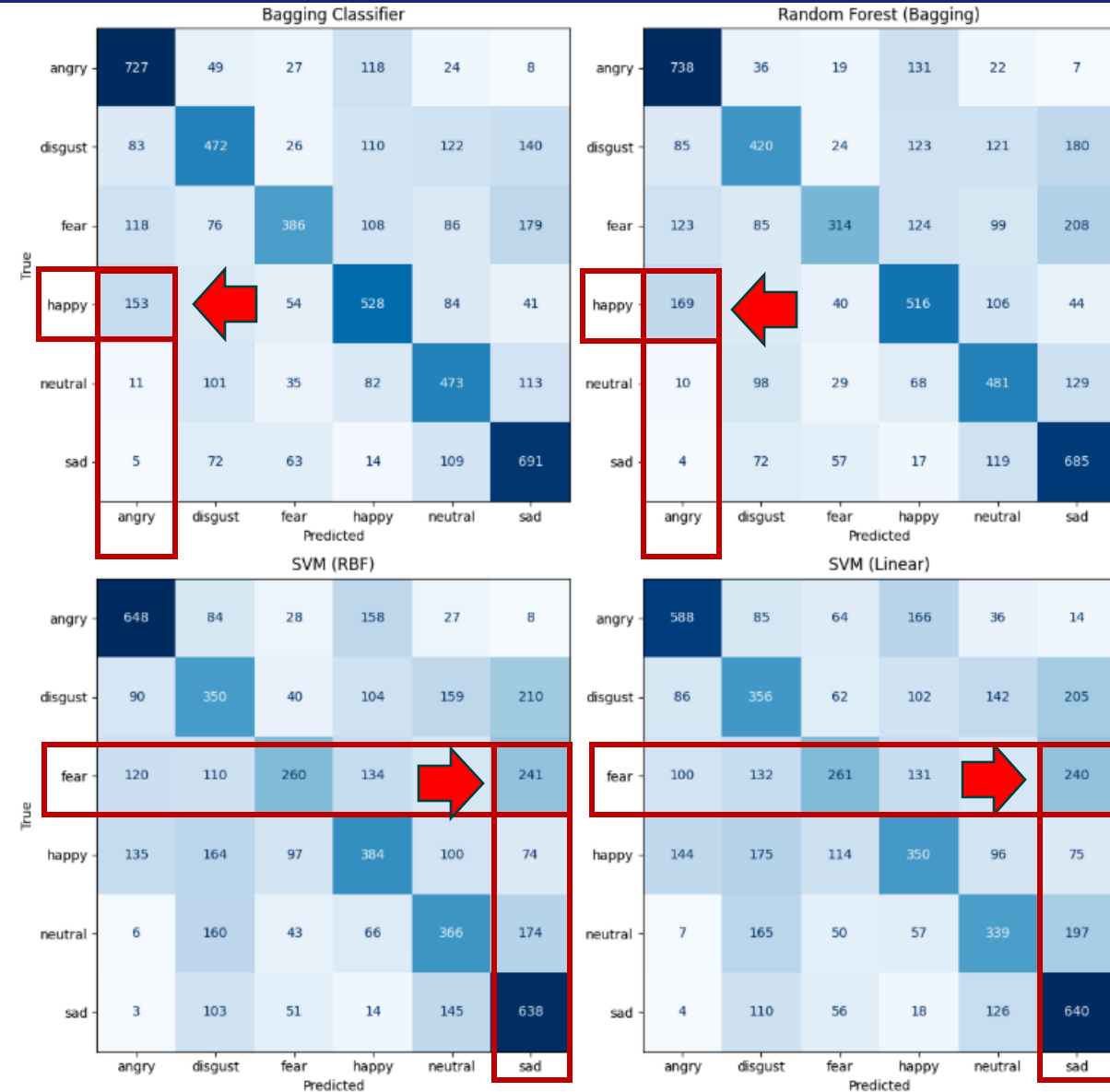
Predicted

Part A – Baseline Performance Insights [5/5]

Error patterns reveal deeper challenges

Observation 2: Different Algorithms Fail Differently

- ! **Bagging Classifier/Random Forest** struggled with **happy-angry** distinctions
- ! **SVM** struggled more with **fear-sad** distinctions
- ! This observation led us to hypothesize that **combining models** might allow their **strengths to compensate for each other's weaknesses**.

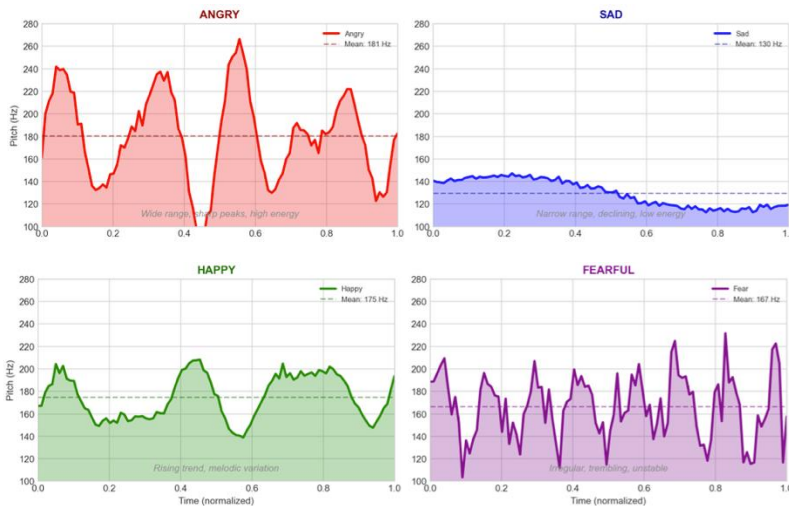


Part B – Key Improvements

Model confuses between these emotion pairs: Fear-Sad, Angry-Happy, Disgust-Sad

#1 Enhanced Feature Engineering (162 -> 210 features)

- ❑ Introduced new features (+48) to detect the **acceleration of change** and **how the pitch is changing**



#2 Introduced Gradient Boosting

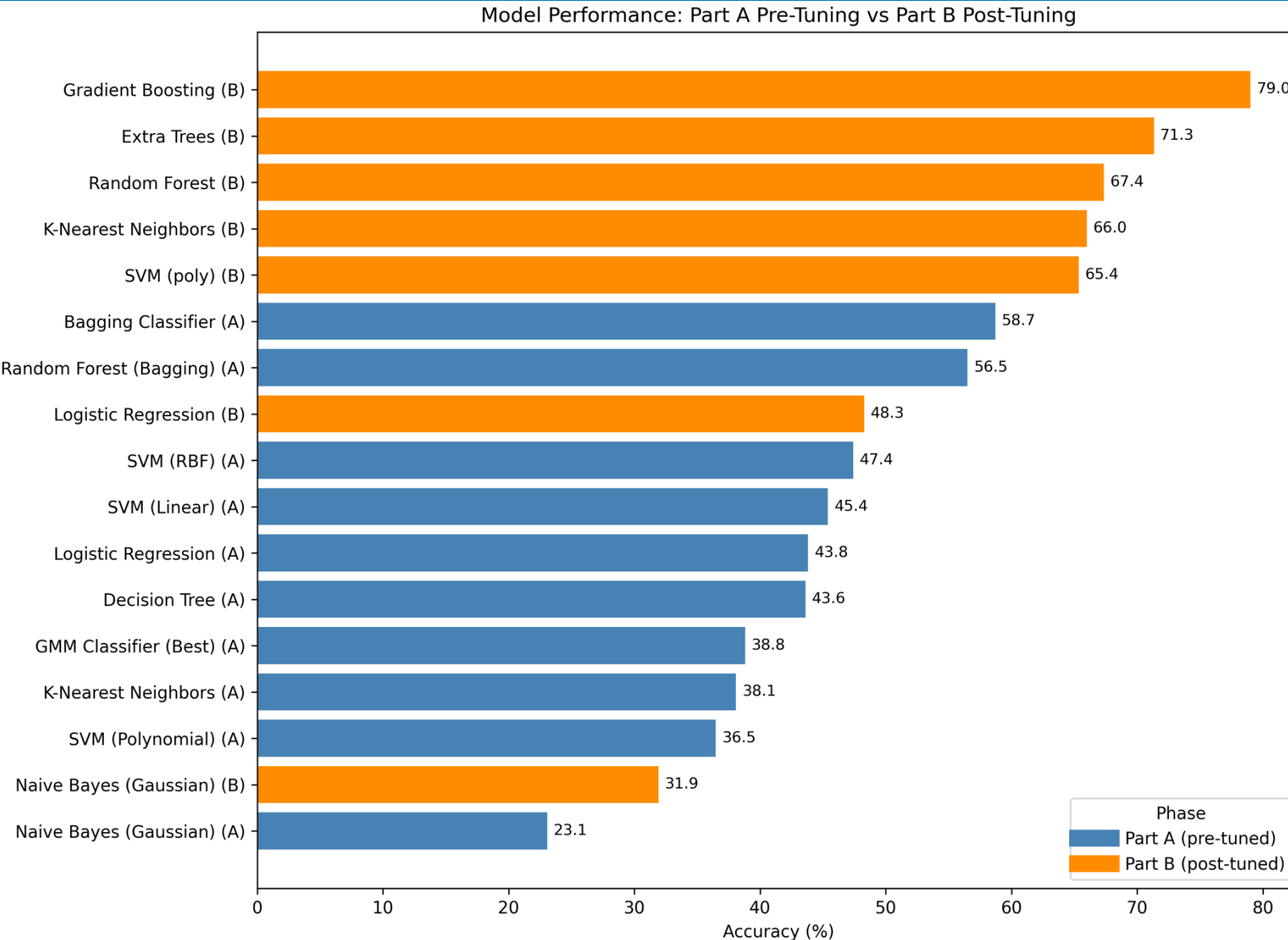
- ❑ **Sequential error-correction** of Gradient Boosting is **perfect** for our use case.
- ❑ Early trees learn the easy distinctions (e.g. loud anger vs quiet neutral)
- ❑ Later trees focus on ambiguous, misclassified samples, **learning subtle differences** in pitch, energy, and spectral patterns.

#3 Hyperparameter Tuning

- ❑ Tuned base models using **RandomizedSearchCV** with **5-fold cross-validation**
- ❑ Chose random search over full grid search because it is **computationally prohibitive**
- ❑ Offers a fast, efficient **trade-off** between **optimality** and **computation time**.

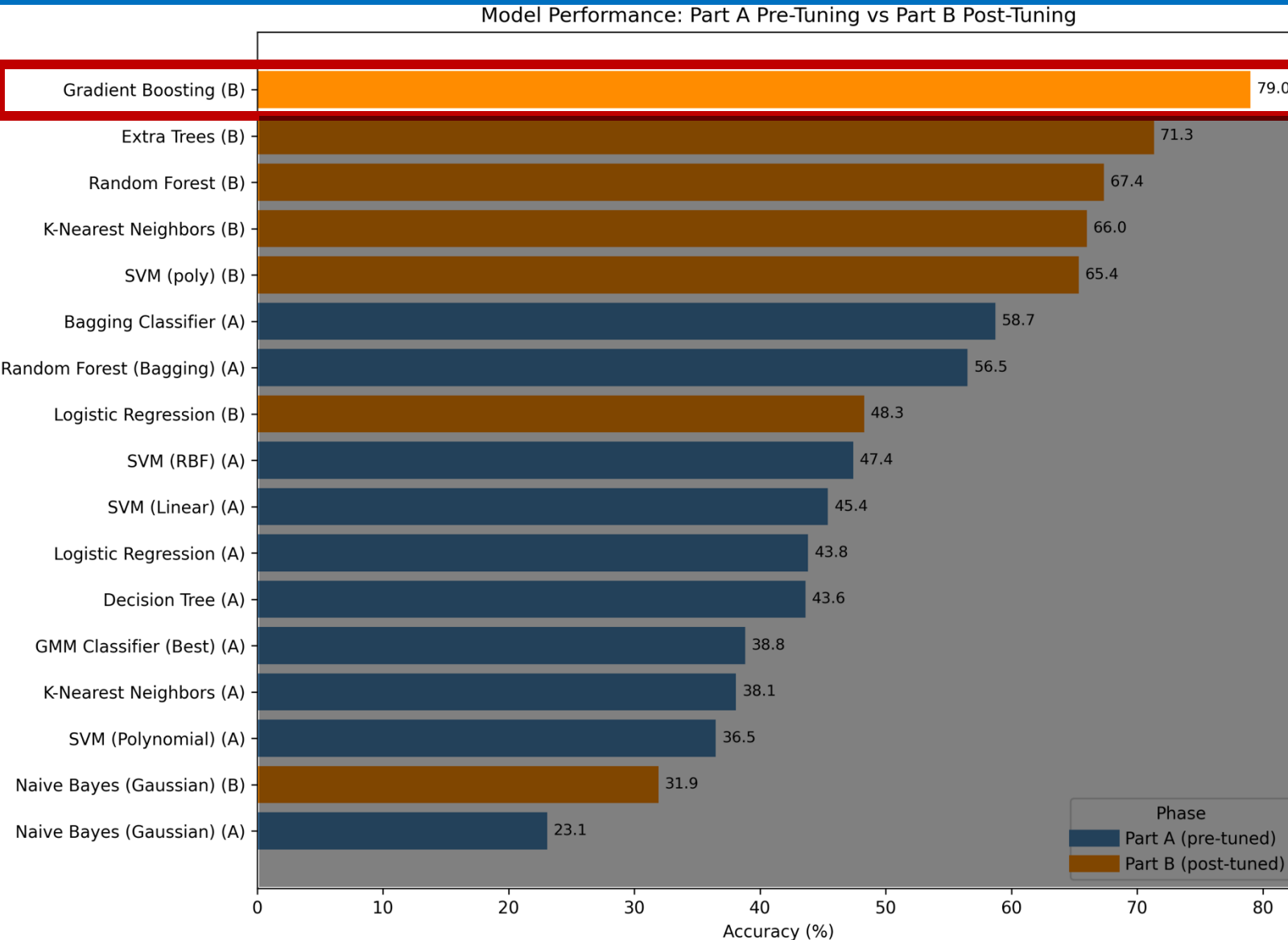
Part B – Key Improvements: Performance Results

Better features, stronger models, deeper tuning



Part B – Key Improvements: Performance Results

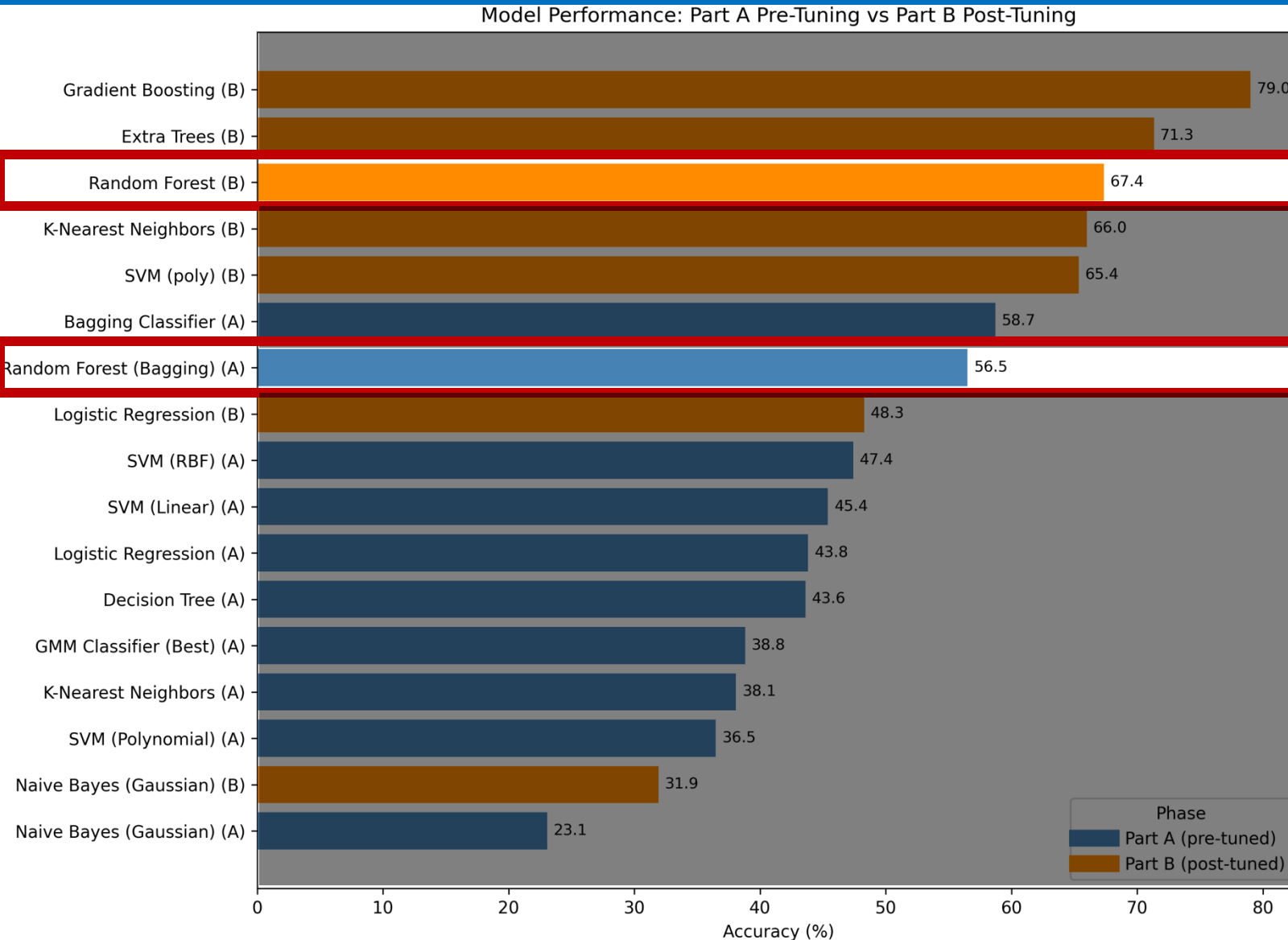
Better features, stronger models, deeper tuning



❑ **Gradient Boosting** became our top performing model at 79% accuracy

Part B – Key Improvements: Performance Results

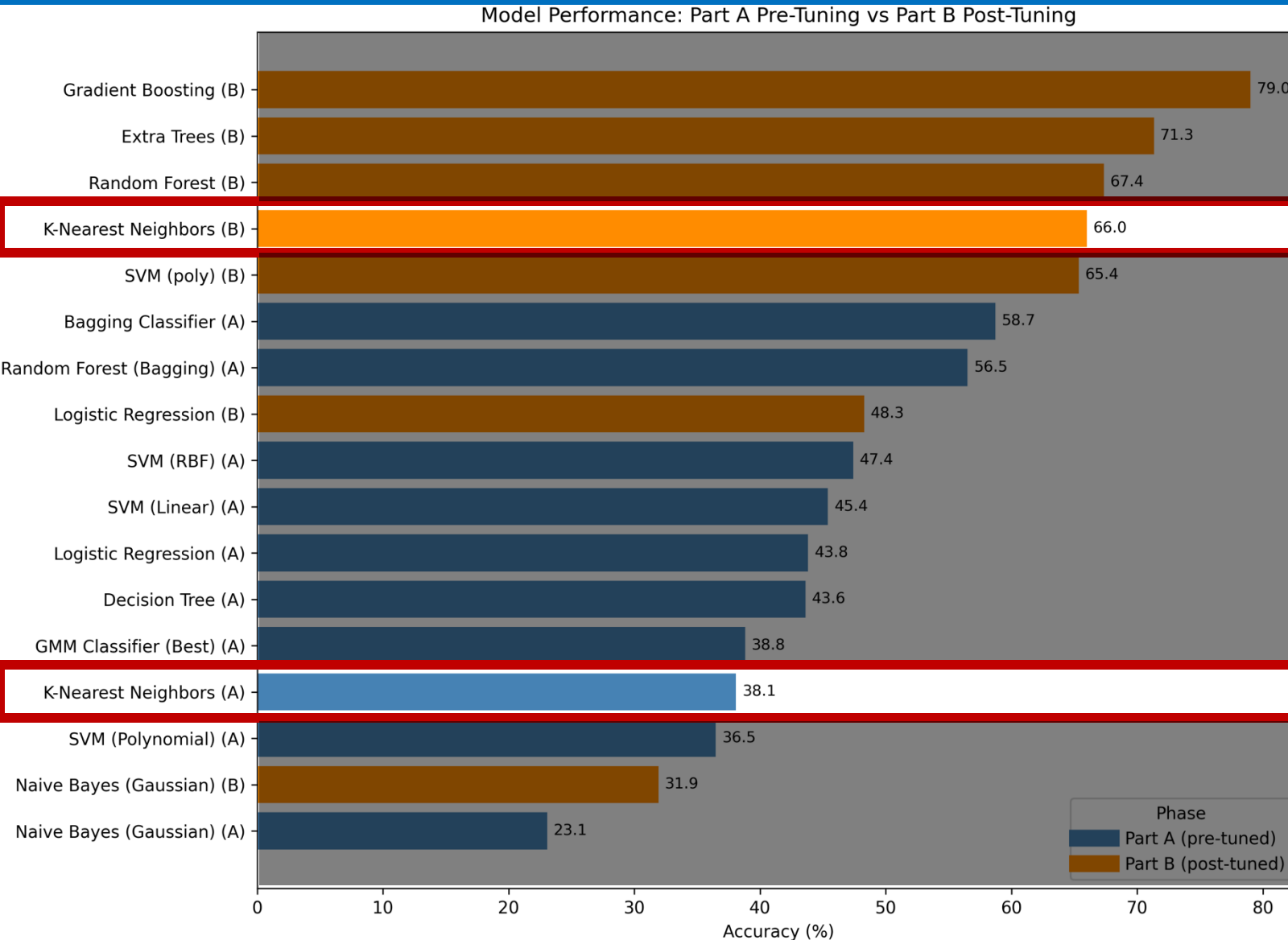
Better features, stronger models, deeper tuning



- ❑ **Gradient Boosting** became our top performing model at 79% accuracy
- ❑ Clear from the results the models are **powerful enough** to **exploit the new data** and the **tuning process** enabled each model to sit at a **near-optimal operating point**

Part B – Key Improvements: Performance Results

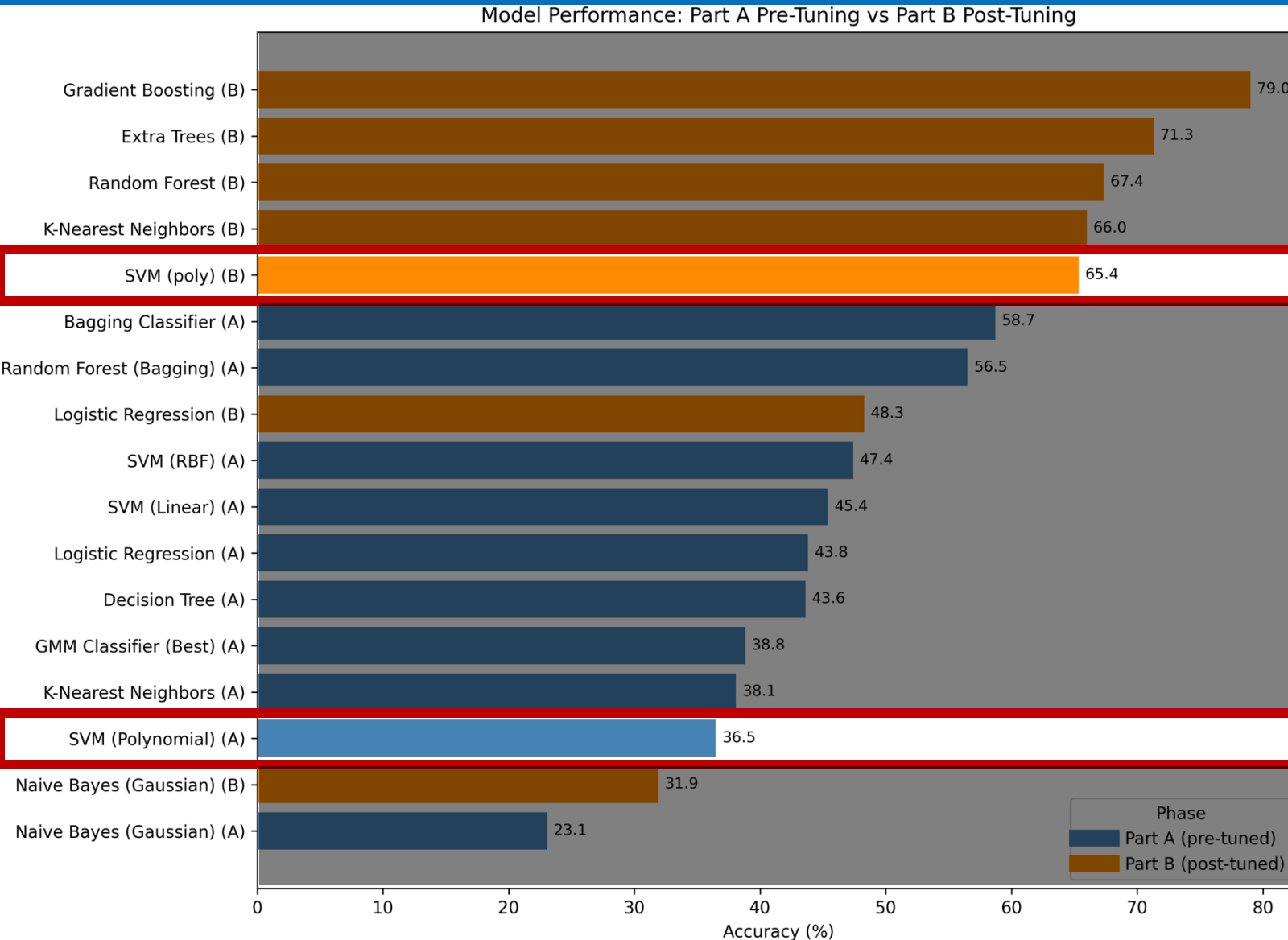
Better features, stronger models, deeper tuning



- ❑ **Gradient Boosting** became our top performing model at 79% accuracy
- ❑ Clear from the results the models are **powerful enough** to **exploit the new data** and the **tuning process** enabled each model to sit at a **near-optimal operating point**

Part B – Key Improvements: Performance Results

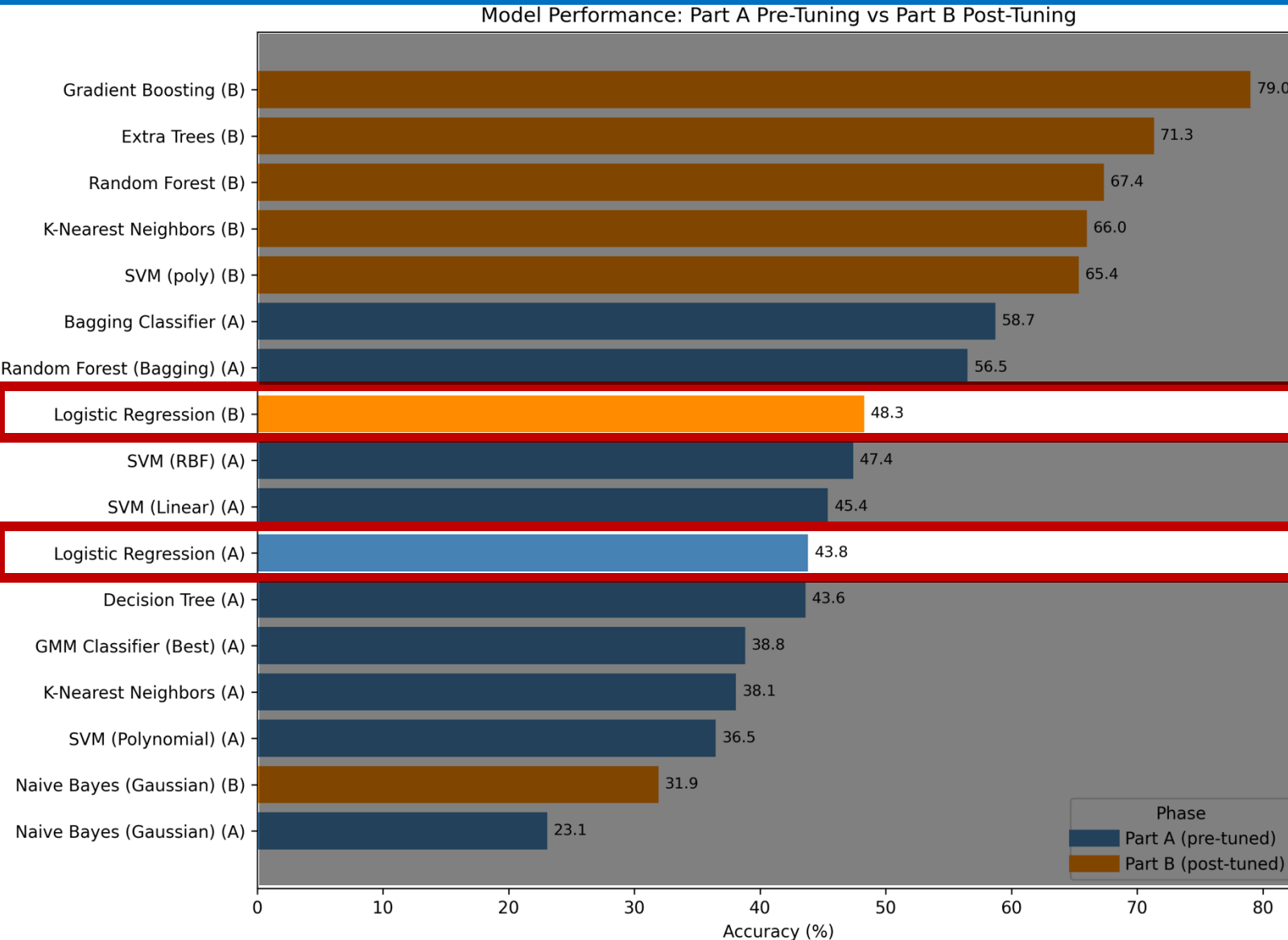
Better features, stronger models, deeper tuning



- ❑ **Gradient Boosting** became our top performing model at 79% accuracy
- ❑ Clear from the results the models are **powerful enough** to **exploit the new data** and the **tuning process** enabled each model to sit at a **near-optimal operating point**

Part B – Key Improvements: Performance Results

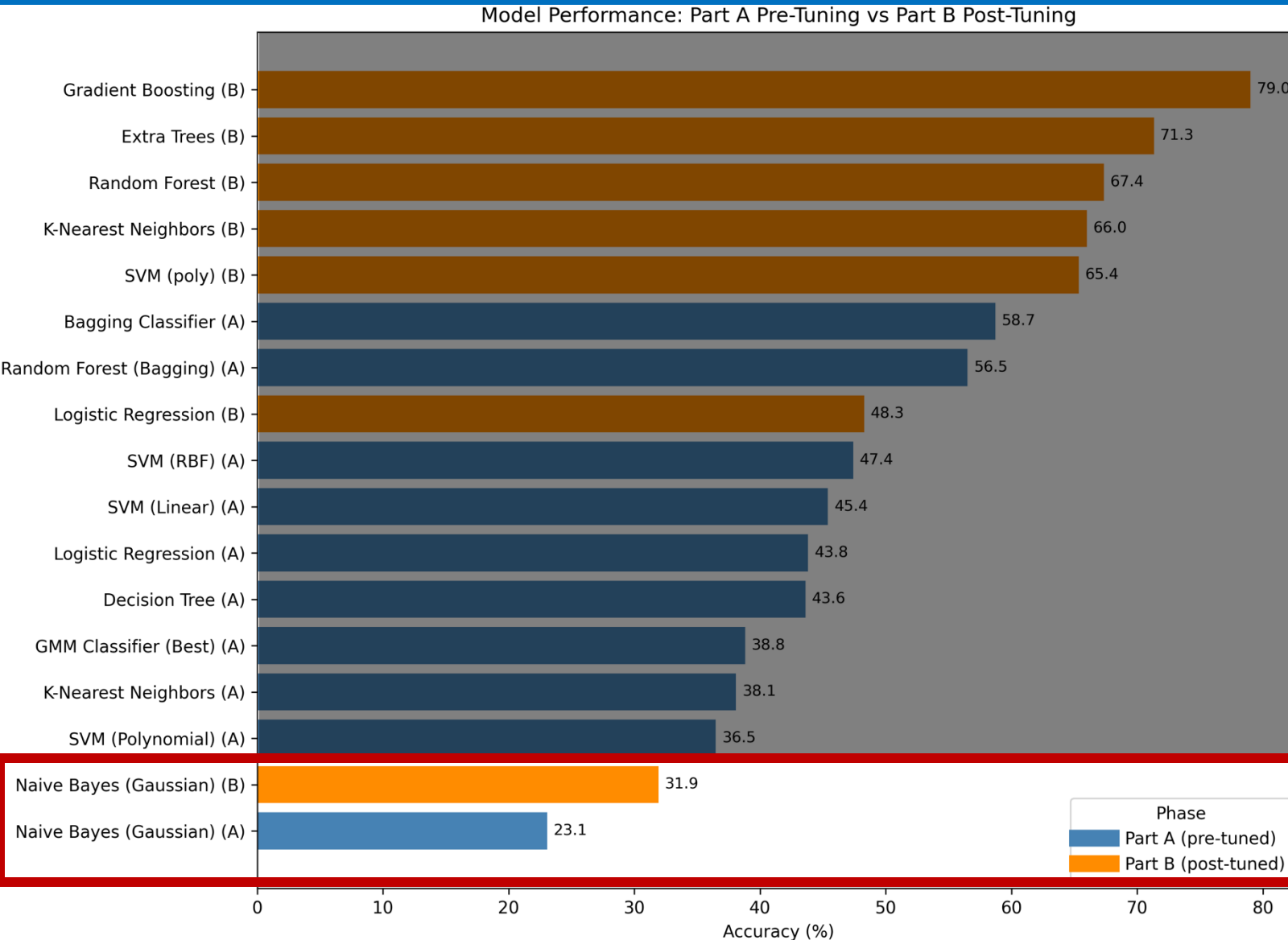
Better features, stronger models, deeper tuning



- ❑ **Gradient Boosting** became our top performing model at 79% accuracy
- ❑ Clear from the results the models are **powerful enough** to **exploit the new data** and the **tuning process** enabled each model to sit at a **near-optimal operating point**

Part B – Key Improvements: Performance Results

Better features, stronger models, deeper tuning

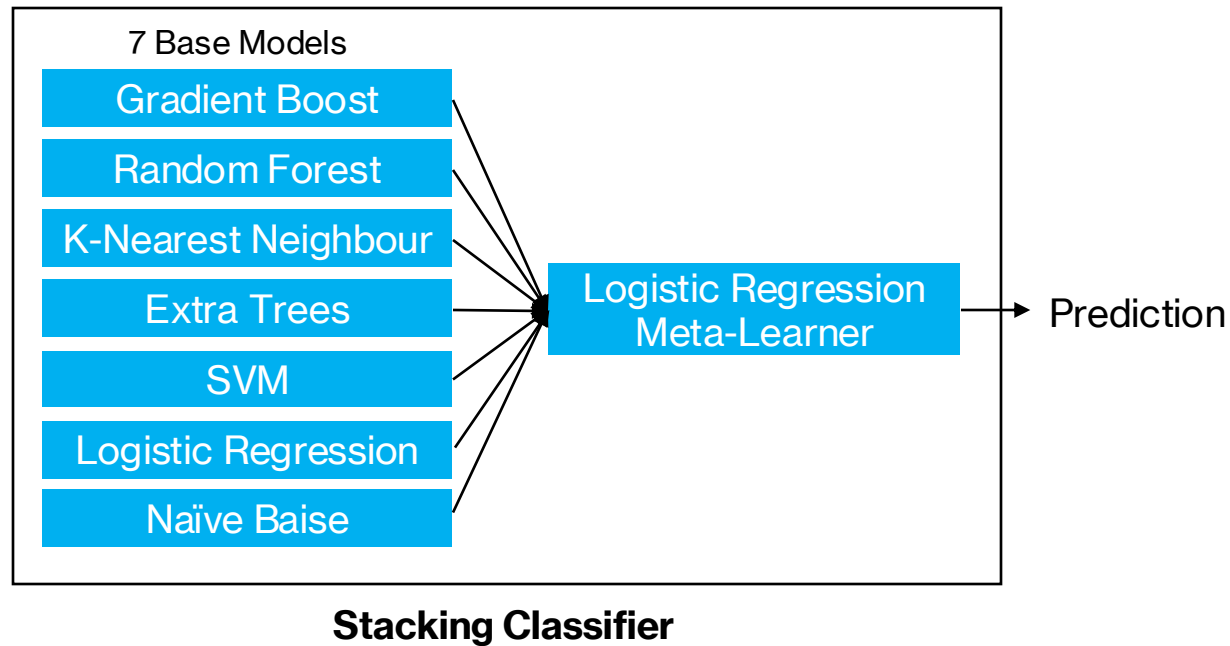


- ❑ **Gradient Boosting** became our top performing model at 79% accuracy
- ❑ Clear from the results the models are **powerful enough** to **exploit the new data** and the **tuning process** enabled each model to sit at a **near-optimal operating point**

Part B – Finding the Optimal Ensemble [1/2]

Seven tuned base models combined into a single-layer stack

Q) Since different models struggled with different emotion pairs, could **combining models** allow their **strengths to compensate** for **each other's weaknesses**?



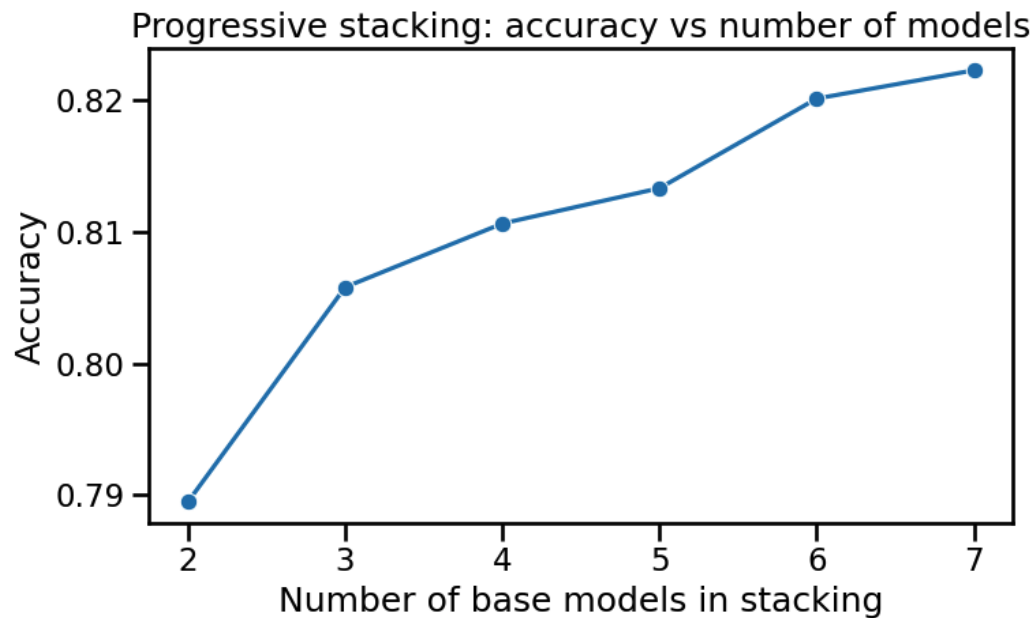
Recent paper shared that **Stacking Classifier** could lead to **superior performance** compared to other traditional machine learning models and even deep learning methods

Prajapati, Y. &. (2025). Systematic Evaluation of Deep Learning Paradigms for Speech Emotion Recognition Using Diverse Audio Sources. *1318-1330*.

Part B – Finding the Optimal Ensemble [2/2]

Seven tuned base models combined into a single-layer stack

Q) Since different models struggled with different emotion pairs, could **combining models** allow their **strengths to compensate** for **each other's weaknesses**?



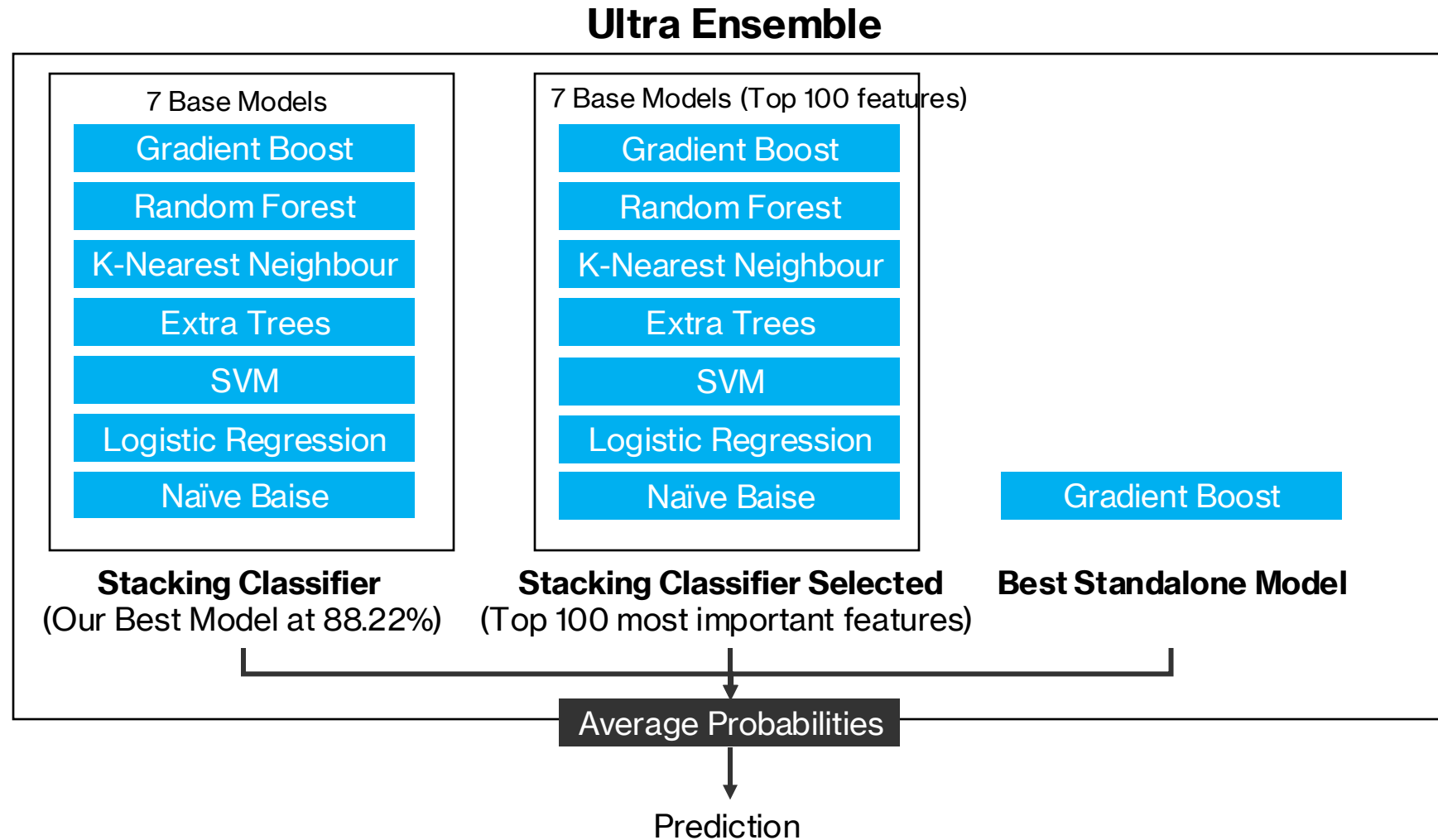
Iter	Models	Accuracy	F1 Wgt
1	gb,rf	78.95%	78.91%
2	gb,rf,knn	80.58%	80.55%
3	gb,rf,knn,et	81.06%	81.04%
4	gb,rf,knn,et,svm	81.33%	81.32%
5	gb,rf,knn,et,svm,lr	82.01%	81.99%
6	gb,rf,knn,et,svm,lr,nb	82.22%	82.20%

Our **Best Model** so far at **82.22%** accuracy!

Even the **weakest model Naïve Bayes** brought a **unique “perspective”** and improved accuracy score

Part B – Can we do better with an “Ultra Ensemble”? [1/2]

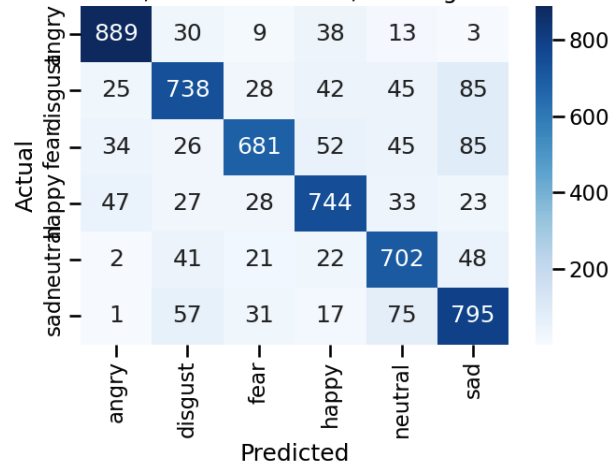
Ensemble of Ensembles



Part B – Can we do better with an “Ultra Ensemble”? [2/2]

Ensemble of Ensembles

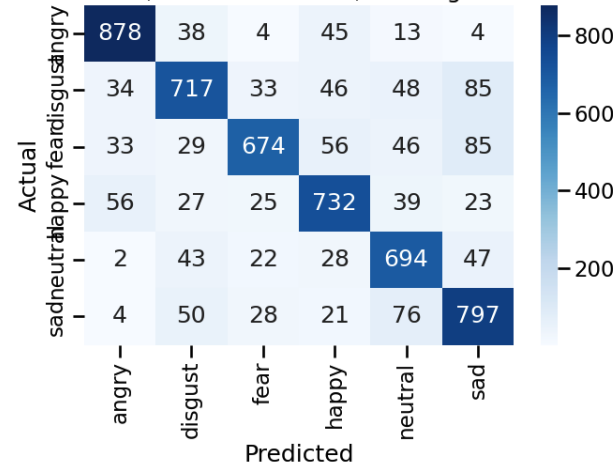
Ultra Ensemble v1 (Best + Best+K, Avg Probabilities)
Acc: 81.49%, F1-macro: 0.814, F1-weighted: 0.815



Stacking+Stacking (Top 100)

81.49% (-0.73%)

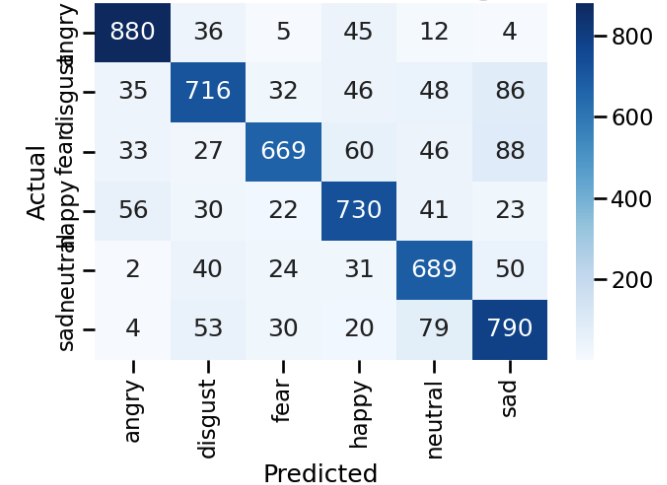
Ultra Ensemble v2 (+ GradientBoosting, Avg Probabilities)
Acc: 80.47%, F1-macro: 0.804, F1-weighted: 0.804



Stacking+Gradient Boosting

80.47% (-1.75%)

Ultra Ensemble v3 (Best + GradientBoosting, Avg Probabilities)
Acc: 80.15%, F1-macro: 0.800, F1-weighted: 0.801



Stacking+Stacking (Top 100)+GB

80.15% (-2.07%)

- ❑ Every combination that included a second layer of ensembling **performed worse** than our best single layer stacking model (**82.22%**)
- ❑ **Conclusion:** Adding more complexity is **not always better**. The second layer of averaging likely diluted the strong, confident predictions from the already highly-optimized single-stacking model

Developing the ML App

Overview

Classroom concepts and tools applied in this app development

Software Development

- ✓ Clean Architecture
- ✓ Test Driven Development
- ✓ Unit, Integration & E2E Tests
- ✓ Static Code Analysis
- ✓ Local Testing on Minikube

DevOps

- ✓ Deployed on Cloud AWS (EKS + Sagemaker)
- ✓ Terraform for IaC (Infra as Code)
- ✓ CI/CD Integration with GitHub
- ✓ GitHub Secrets Management

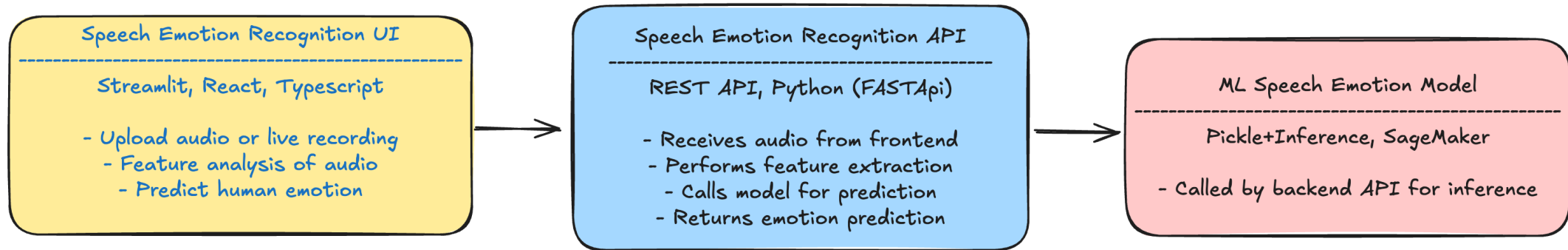
ML Ops & Observability

- ✓ Model Deployment on AWS Sagemaker
- ✓ Evidently AI Integration for Model Performance Monitoring
- ✓ Model Versioning
- ✓ App Performance & EKS Monitoring on Prometheus & Grafana

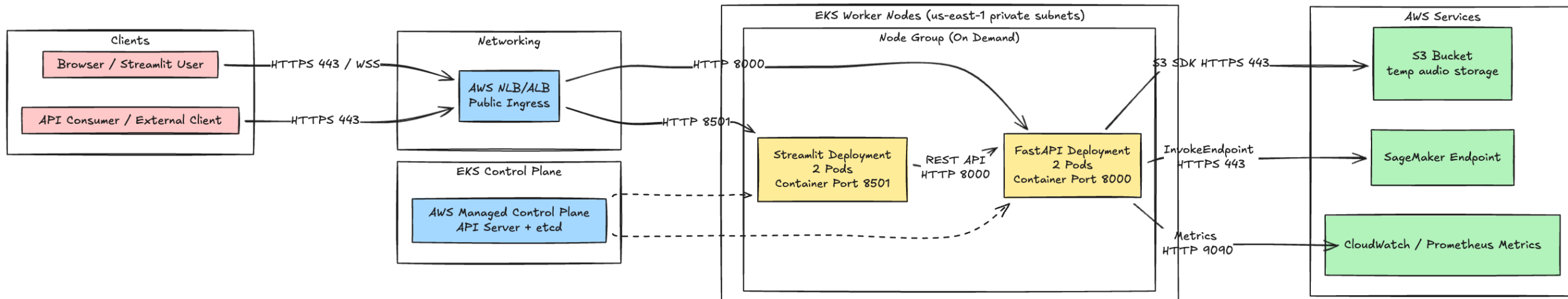
Application Architecture

Container and Deployment Diagrams depicting high level application setup

Container Diagram

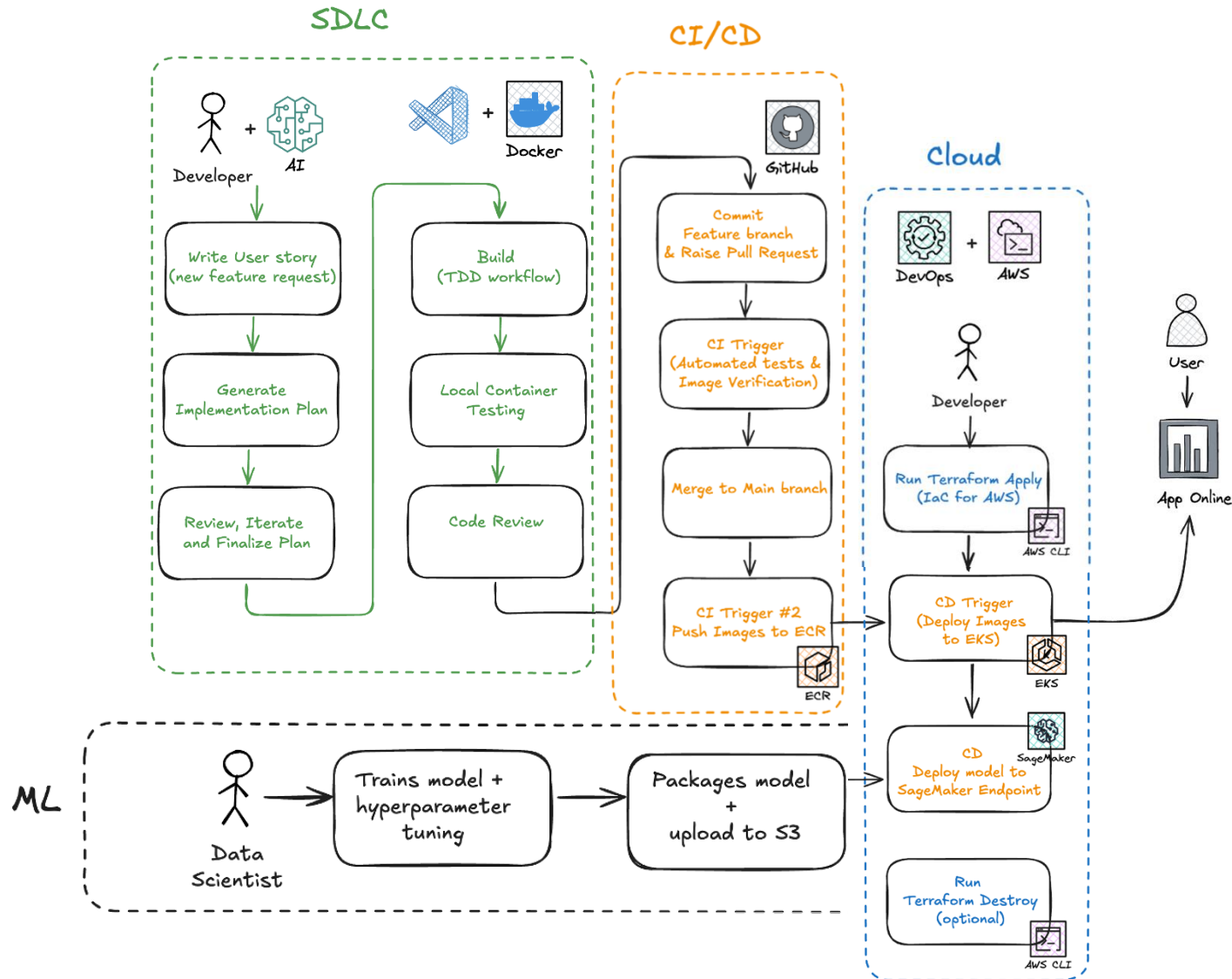


Deployment Diagram



Model Development Lifecycle with SDLC

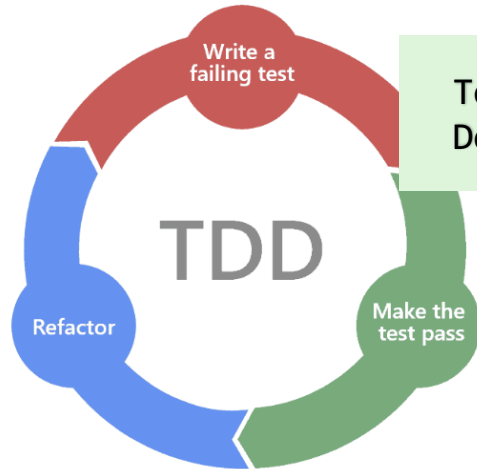
Overall ML Pipeline used for Speech Emotion Recognition model and app development



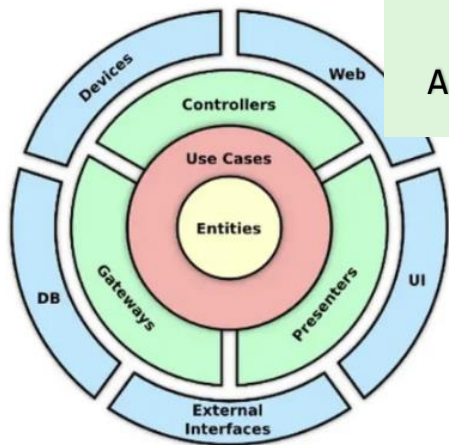
The Build Phase

Iterative App Development, Testing & Local Deployment

★ Build

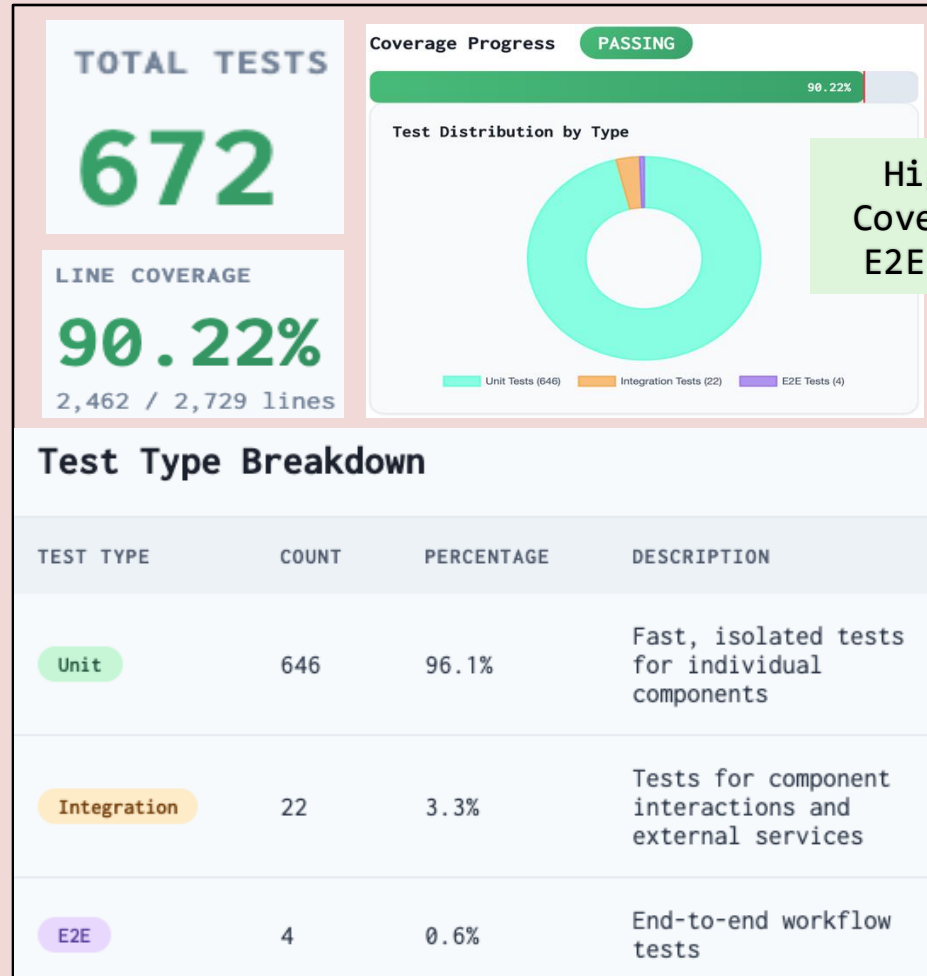


Test Driven Development



Clean Architecture

★ Test



High Test Coverage and E2E Testing

★ Deploy

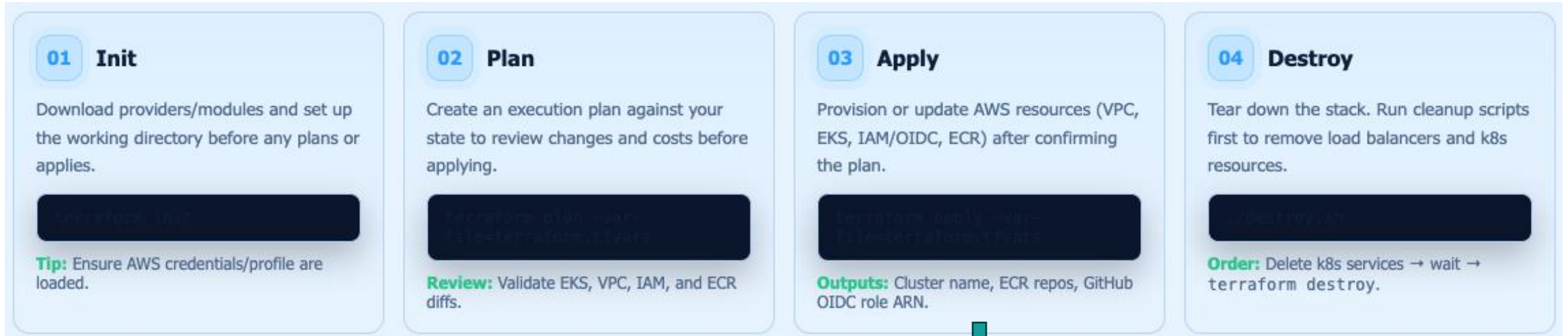


minikube

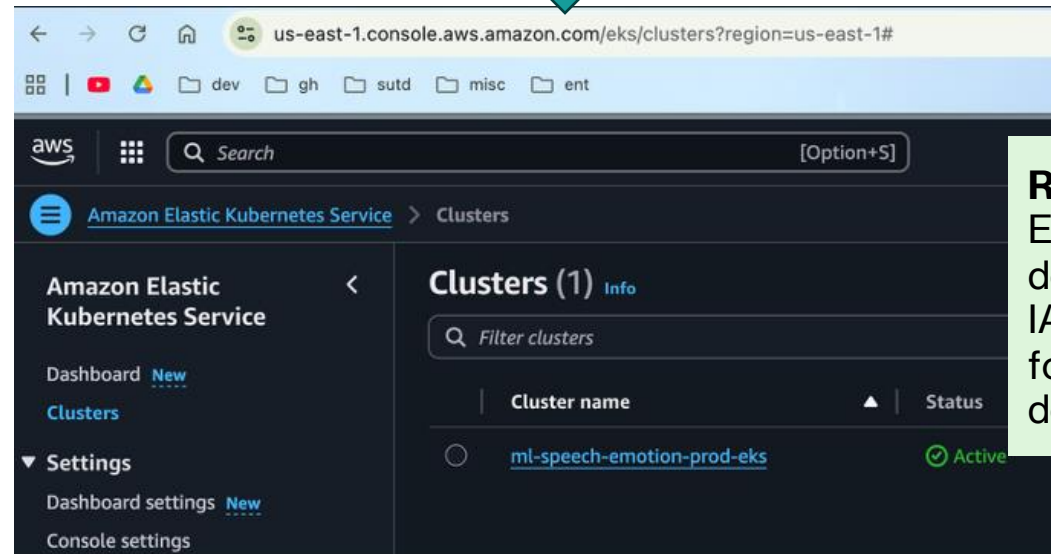
Local simulation of EKS deployment

EKS Infrastructure Setup using Terraform

(IaC) Deployment Lifecycle



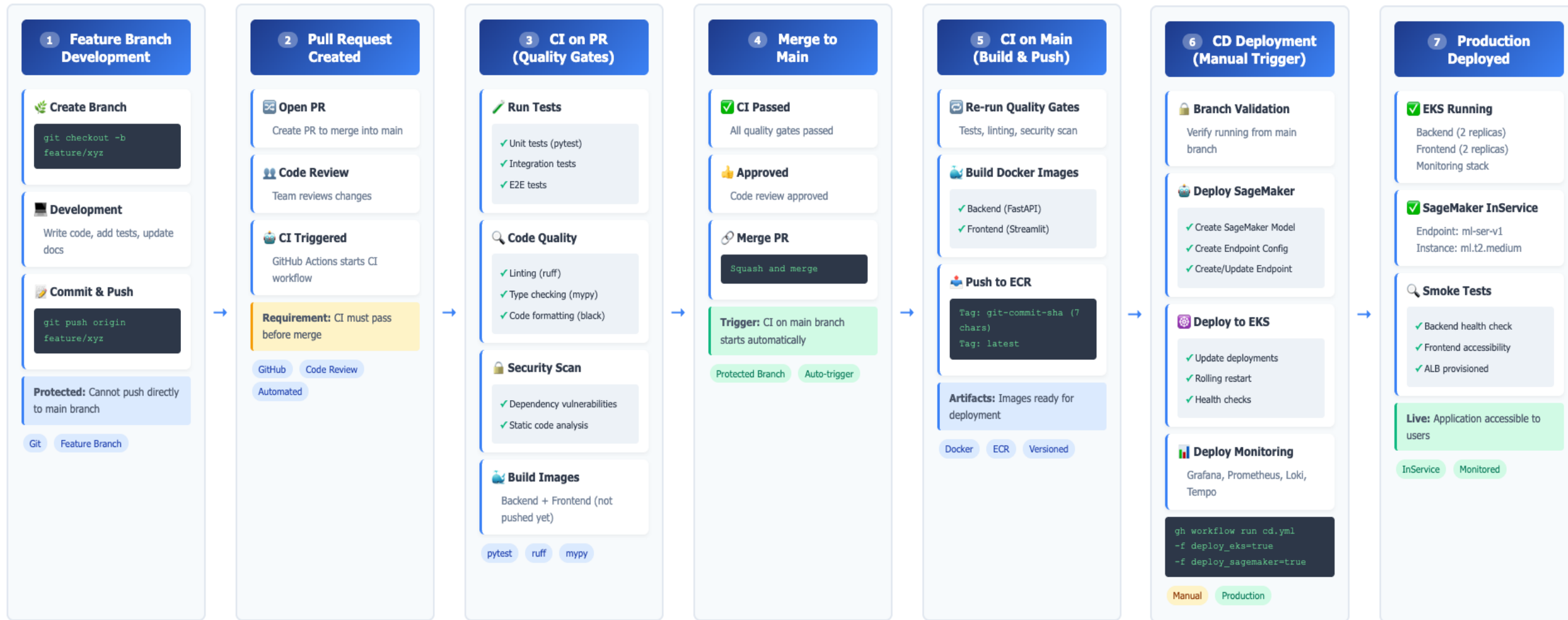
 **Benefit: Infrastructure stored as code. Highly reproducible and deterministic upon every deployment!**



Result:
EKS Infrastructure deployed together with all IAM permissions required for subsequent application deployment

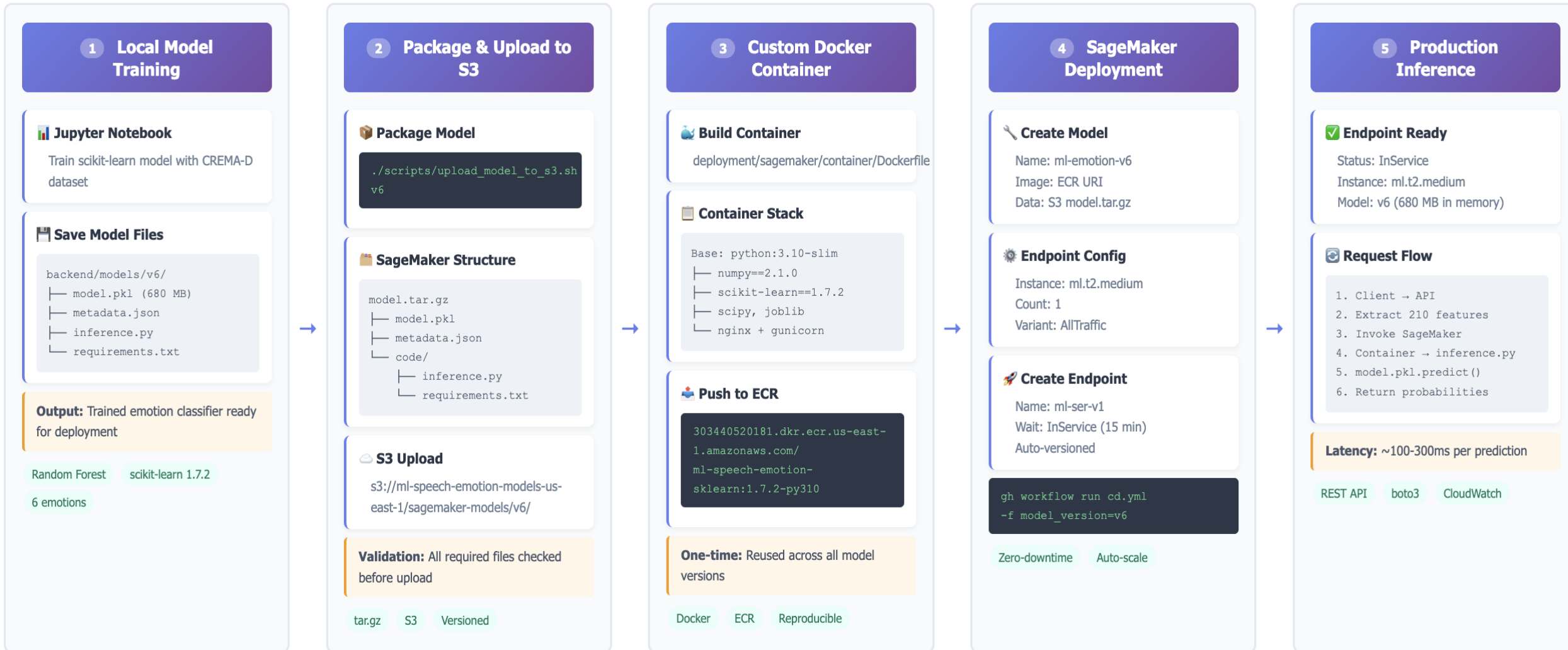
Application Deployment using CI/CD

DevOps Lifecycle used for ML App deployment



Model Deployment- Sagemaker

Deployment Steps adopted for automated model deployments to AWS Sagemaker endpoint



Model Performance Monitoring using Evidently AI

Backend Integration with Evidently AI to monitor model performance



Introduced capability to record user feedback on actual emotion to generate real-world dataset for future model retraining based on drift detection

Overall Model Performance

Classification quality (latest)

Accuracy	Precision	Recall	F1 Score
38.3%	34.3%	31.5%	31.7%

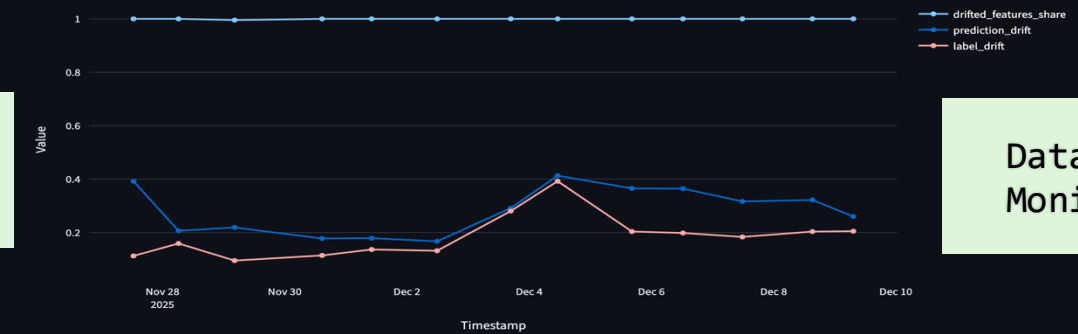
Performance Over Time



Drift signals (latest)

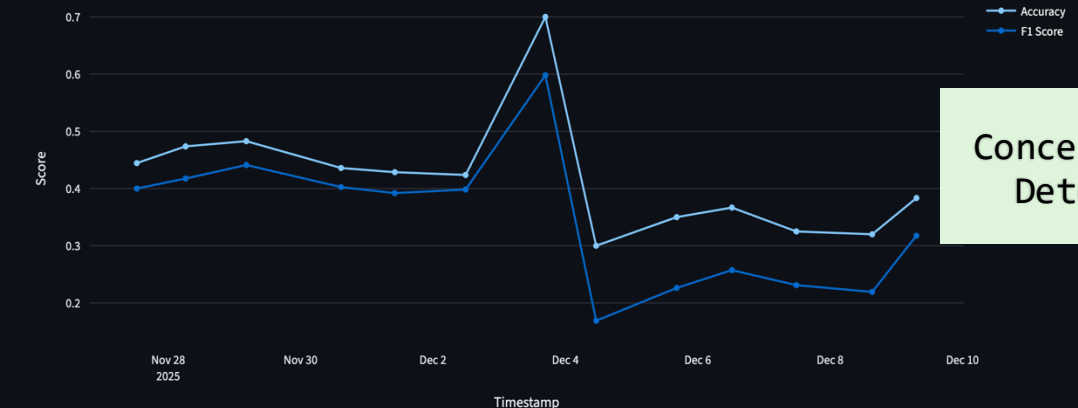
Drifted Features (%)	Drifted Features (#)	Prediction Drift (p-value)	Label Drift (p-value)
100.0%	212.0	0.260	0.205

Drift Trends Over Time



Data Drift Monitoring

Concept Drift Detection



Concept Drift Detection

Observability

Deployed Monitoring Stack to EKS with Loki, Promtail, Prometheus & Grafana



Kubernetes Dashboard monitors EKS nodes, CPU, Memory and Disk I/O Health.



Application Dashboard monitors API requests health, streamlit and backend logs for anomalies

